

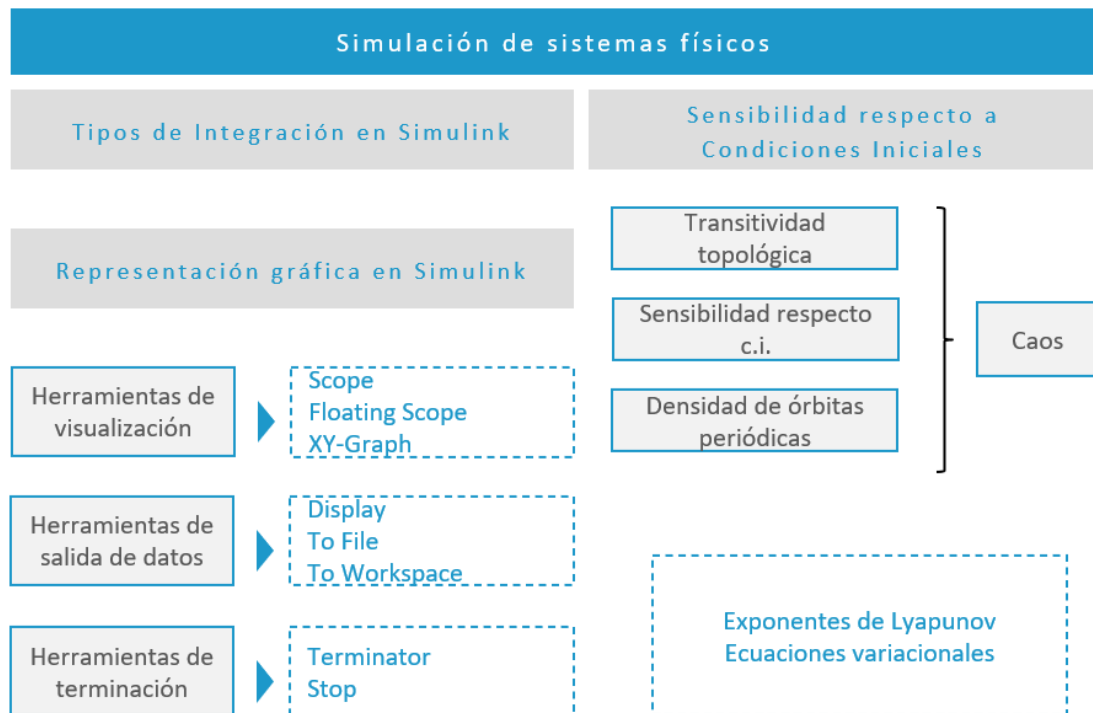
Modelado de Sistemas Dinámicos

Simulación de Sistemas Físicos

Índice

Esquema.	2
Ideas clave	3
4.1 Introducción y objetivos	3
4.2 Resultados de simulación	3
4.3 Sensibilidad respecto condiciones iniciales	9
4.4 Simulink: herramientas para la representación de datos.	12
4.5 Modelos en Simulink.	15
4.6 Referencias bibliográficas	19

Esquema



4.1 Introducción y objetivos

En temas anteriores hemos visto las bases del modelado y de Simulink. También hemos aprendido a introducir ecuaciones diferenciales en dicho programa.

En este tema hablaremos de los diferentes resultados que podemos esperar de la simulación. También hablaremos de las diferentes formas de representación de señales en Simulink y exploraremos las formas de introducir condicionales y condiciones de parada.

También hablaremos de la sensibilidad respecto a condiciones iniciales y el caos.

Terminaremos el tema viendo varios ejemplos de sistemas físicos implementados en Simulink:

- ▶ Condiciones de parada en la bala de cañón.
- ▶ Péndulo doble.
- ▶ El atractor de Lorentz.

4.2 Resultados de simulación

El objetivo de toda simulación es obtener los resultados deseados. El objetivo para esta sección es exponer todos aquellos conceptos que debemos tener en cuenta en la obtención de los resultados.

El primer concepto que debemos tener en cuenta es cuál es el objetivo de nuestra simulación. Al realizar una simulación podemos tener como objetivo:

- ▶ Conocer la evolución de un sistema. El primer objetivo que se nos puede ocurrir pa-

ra realizar una simulación es conocer cuál será la evolución de un sistema. Esto es, saber qué condiciones pueden tomarse y cuáles son factibles. Esto nos puede permitir realizar predicciones del sistema o buscar regiones de las variables de estado en las que es más (o menos) probable que nos encontremos.

- ▶ Afinar los parámetros de un sistema. Si nuestro objetivo es conocer cuáles son los parámetros que debemos usar en nuestro sistema podemos realizar simulaciones y comparar con el valor obtenido en el experimento real. Otra alternativa será fijar los parámetros de diseño del sistema para conseguir un resultado concreto.
- ▶ Obtener valores óptimos para un sistema. En este tipo de simulación se pretende encontrar la mejor solución posible en cuanto a alguno de los parámetros del sistema. Ya sea encontrar la solución que alcanza una cierta posición en el menor tiempo o cumpliendo un cierto objetivo a nivel de alguna de las variables directas o derivadas del sistema. Cuando tenemos este objetivo, tenemos que tener en cuenta que la simulación no nos dará la solución, sino que deberemos usar una herramienta adicional en la que se incluya la simulación.

Una vez tenemos claros los objetivos de nuestra simulación debemos plantearnos qué tipo de variable deseamos. Podemos estar interesados en únicamente el valor final, por ejemplo, qué temperatura tendrá un elemento determinado después de tres horas o conocer la presión atmosférica una semana más tarde. En este caso únicamente nos interesará el estado final y no el recorrido. Sin embargo, puede que nos interese saber la trayectoria a lo largo de toda la simulación, por ejemplo para saber si ha colisionado con algún objeto o alguna región prohibida. En este caso, nos interesará conocer los estados a lo largo de toda la simulación.

En el caso en el que queramos conocer únicamente el resultado final de la simulación podemos emplear el bloque `Display`. Este bloque nos permitirá visualizar el resultado de la señal que recibe a medida que evoluciona la simulación. Usualmente las simulaciones serán muy rápidas. Por este motivo solo veremos el resultado final. Por defecto, todas las simulaciones terminan en 10 unidades de tiempo. Este valor lo podemos modificar en la parte superior de la pestaña de simulación al valor deseado. De esta forma, podemos regular el tiempo de integración.

Cuando realizamos simulaciones numéricas, un otro concepto a tener en cuenta es el tipo de integrador o propagador numérico que empleamos. Algunos métodos clásicos incluyen el método de Runge-Kutta, el de Adams-Bashfort, métodos simplécticos o métodos de Taylor. Cada método tiene sus ventajas y sus inconvenientes. Todos los métodos tienen asociado un orden. Dicho orden hace referencia a la capacidad que

tienen de realizar una buena predicción de la solución. Por norma general, a mayor orden, mayor precisión pero menor velocidad. En Simulink tenemos la capacidad de elegir el método de integración que queremos emplear dentro de una lista extensa de métodos. Para ello, deberemos abrir la ventana de configuración de la simulación de Simulink (pestaña *Modeling* → *Model Settings* → *Model Settings*, o mediante el atajo **Ctrl+e**). De aquí nos interesará la configuración del *Solver*. En la parte superior podremos escoger los tiempos inicial *Start time* y final *Stop time*. En el siguiente bloque de configuración nos dejará escoger el tipo de integrador que queremos emplear. Los detalles de estos métodos escapan al objetivo de esta asignatura. Sin embargo conviene conocer las opciones de las que se disponen. Podremos escoger entre:

- ▶ Integradores de paso fijo *Type: Fixed Step*. Los integradores de paso fijo realizan un paso de integración temporal cada unidad fijada de tiempo. Entre otras opciones podemos encontrar el método de Euler (orden 1), el método de Heun (orden 2), el método de Runge-Kutta (orden 4), o el de Dormand-Prince (órdenes 5 y 8). A medida que el orden del método incrementa podremos escoger valores mayores para el paso.
- ▶ Integradores de paso variable *Type: Variable-Step*. Los integradores de paso variable realizan algunas operaciones adicionales para determinar el mejor paso para continuar con la propagación del método. En Simulink podemos encontrar, entre otros, los métodos de Bogacki-Shampine (orden 2-3), Dormand-Prince (orden 4-5) o Adams (órdenes 1 a 13). También dispondremos de métodos para la resolución de sistemas rígidos.

Ejercicio 1. Considera un centro lineal dado por las ecuaciones diferenciales:

$$\begin{cases} \dot{x} = -y \\ \dot{y} = x \end{cases}$$

Realiza varias simulaciones con distintos integradores y tiempo final $T = 300$ y observa las diferencias que se obtienen.

Indicación: Recuerda que en un centro lineal la norma $\|(x, y)\|$ es constante.

En este caso, el modelo es muy sencillo de implementar. Necesitamos dos bloques de integración, y un bloque de producto por escalar para cambiar el signo. Además, vamos a añadir un bloque de gráfica X-Y para visualizar el resultado. Programaremos el primer integrador correspondiente a las condiciones iniciales de x en 0 y el segundo integrador con condición inicial $y_0 = 1$.

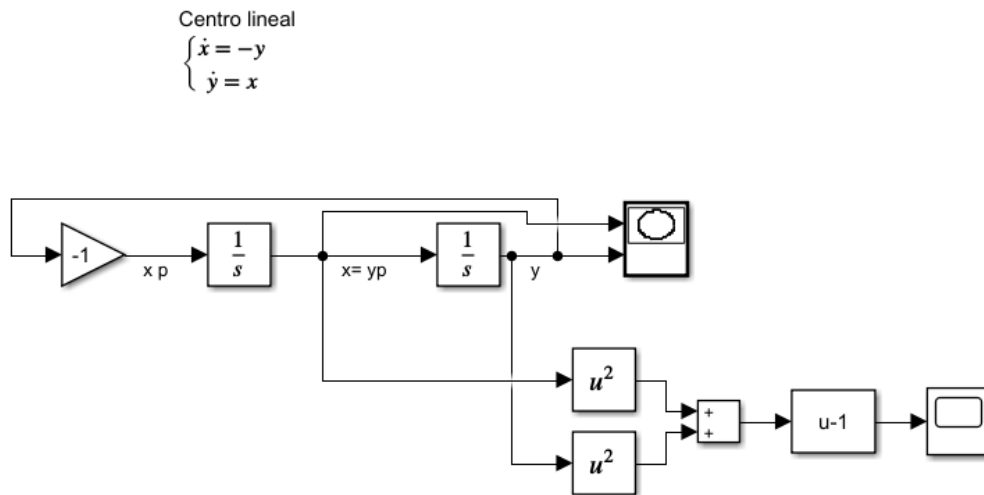


Figura 1: Representación en Simulink del centro lineal y los datos necesarios para representar los errores del sistema.

Además, como sabemos que la norma del vector (x, y) es constante también lo será su cuadrado. Así pues, añadimos dos bloques de elevar al cuadrado y un bloque de suma para poder ver la evolución del error. Finalmente, como consideramos las condiciones iniciales $(x_0, y_0) = (0, 1)$ sabemos que la norma al cuadrado será constante a 1, por lo que añadimos un bloque de bias para ver el error que cometemos. El bloque final diagrama de Simulink final nos queda como se muestra en la Figura 1

A continuación vamos a probar diferentes integradores para evaluar su comportamiento:

- ▶ Primero ejecutaremos la opción estándar, esto es un integrador de paso variable con selección de método automática y precisión de 10^{-5} . Si realizamos la simulación obtendremos los resultados de la Figura 2. Podemos ver como a simple vista la simulación reproduce una circunferencia, como era de esperar. Sin embargo, el error va creciendo hasta llegar a $7 \cdot 10^{-4}$. Veremos que esta es la tónica general. Si nos fijamos en el integrador escogido (esquina inferior derecha de la ventana de nuestro modelo) veremos que el integrador escogido ha sido el definido en la función ode45.
- ▶ A continuación forzaremos algún integrador concreto. Seleccionaremos el método ode113, que sigue un método de Adams. En dicho método se selecciona de forma automática no solo el paso sino también el orden entre 1 y 13. Este método suele ser eficiente y dar buenos resultados. Mantenemos la tolerancia del método a

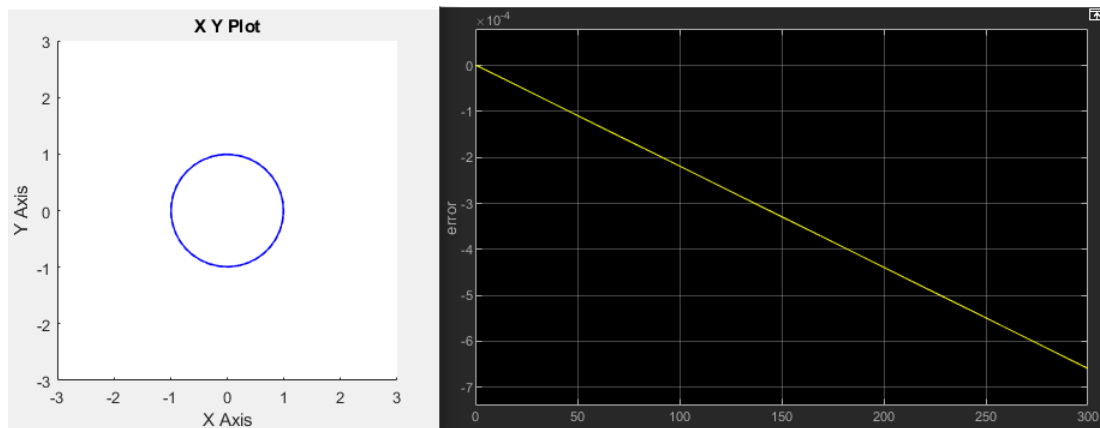


Figura 2: Resultados de simulación con un método de paso variable automáticamente seleccionado y tolerancia de 10^{-5} . Izquierda: representación en las coordenadas x e y . Derecha: error cometido con respecto al radio de la circunferencia ($e = x^2 + y^2 - 1$).

10^{-5} . Después de realizar la simulación obtenemos los resultados de la Figura 3. Vemos en este caso como debido a las operaciones del método el error también va creciendo, sin embargo no lo hace de forma monótona como lo hacía en el caso anterior, sino que realiza pequeñas fluctuaciones. En cuanto a la figura observada, sigue siendo la misma. Esto se debe a que no somos capaces de ver ninguna gran diferencia.

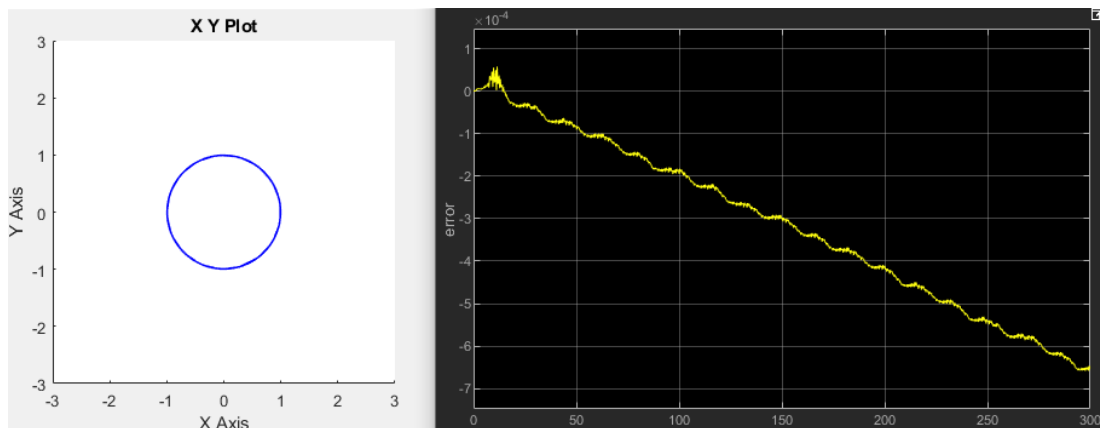


Figura 3: Resultados de simulación con un método de paso variable fijado de ode113 y tolerancia de 10^{-5} . Izquierda: representación en las coordenadas x e y . Derecha: error cometido con respecto al radio de la circunferencia ($e = x^2 + y^2 - 1$).

- Realizaremos ahora un ejemplo con un método de paso fijo. en este tipo de métodos no fijamos una tolerancia, sino la longitud del paso que queremos realizar. Vamos a empezar fijando un paso de 0.01 que es el automático. Una vez tenemos el integrador debidamente configurado vamos a ejecutar la simulación. Obtendremos los resultado que se muestran en la figura 4. Observamos que el tiempo de

ejecución del método es ostensiblemente más largo, tardamos más en obtener los resultados. Con este paso, conseguimos un error máximo de 10^{-5} lo que es algo mejor que el obtenido en los métodos anteriores. Sin embargo, hemos pagado en el precio. Fijémonos que en el caso anterior hemos pedido una tolerancia de 10^{-5} , por lo que será difícil pedir superar dicha tolerancia. El método que se ha seleccionado de forma automática es un método ode3.

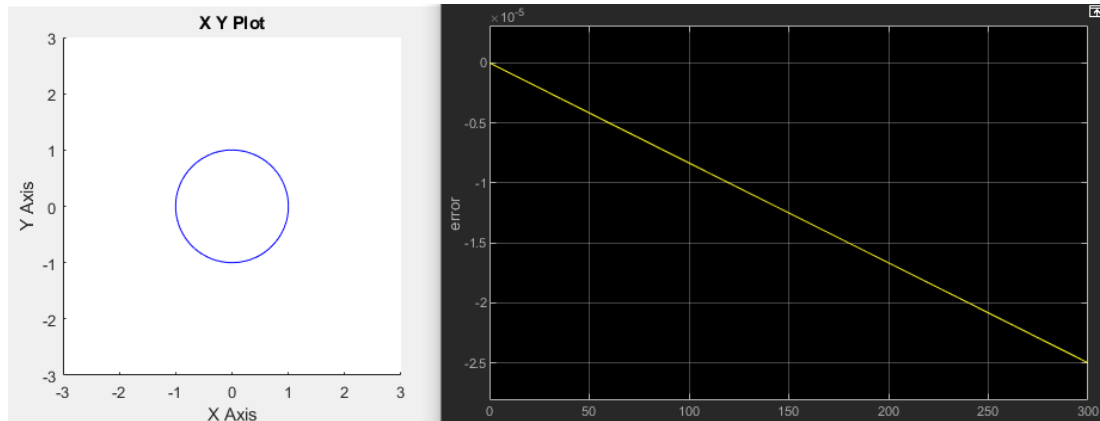


Figura 4: Resultados de simulación con un método de paso fijo de 0.01 y selección del método automática. Izquierda: representación en las coordenadas x e y . Derecha: error cometido con respecto al radio de la circunferencia ($e = x^2 + y^2 - 1$).

- Terminaremos este ejercicio seleccionando un método de Euler con paso de 0.01. Sabemos que Euler es un método de orden 1 y no debería comportarse especialmente bien. Después de seleccionar el paso y el método ejecutamos la simulación. Obtenemos el resultado que se muestra en la Figura 5. En dicha figura podemos observar como el error crece ostensiblemente hasta las 20 unidades de diferencia. Además, el resultado en el plano $X - Y$ también deja de ser un círculo para mostrarse como una espiral que va creciendo en radio. Esto nos demuestra la importancia de escoger un buen método de integración.

Usualmente, Simulink seleccionará el mejor integrador de forma automática. Sin embargo, cuando realicemos nuevos modelos debemos ser conscientes de los errores que pueden provocarse en nuestra simulación debido al integrador. Simulink también nos ofrece la herramienta *Solver Profiler* que nos permitirá realizar el análisis de la simulación. En el vídeo “El *Solver Profiler* de Simulink” veremos cómo utilizarlo y los detalles más importantes del mismo.



Accede al vídeo: El *Solver Profiler* de Simulink.

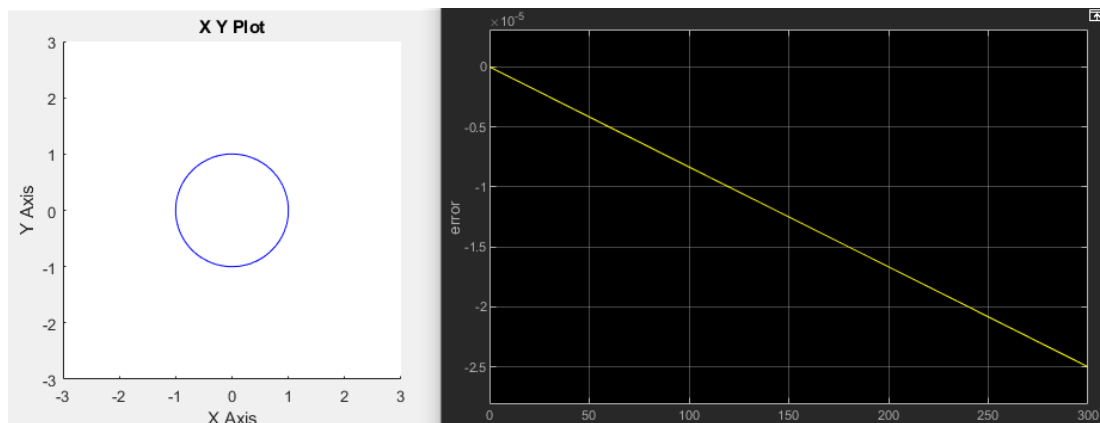


Figura 5: Resultados de simulación con el método de Euler con un paso fijo de 0.01. Izquierda: representación en las coordenadas x e y . Derecha: error cometido con respecto al radio de la circunferencia ($e = x^2 + y^2 - 1$).

4.3 Sensibilidad respecto condiciones iniciales

Uno de los factores que debemos estudiar cuando trabajamos en modelado de sistemas y en sus simulaciones es la sensibilidad respecto de las condiciones iniciales y los parámetros del sistema.

Este fenómeno trata de estudiar como pequeñas diferencias o desviaciones en las condiciones iniciales pueden derivar en grandes diferencias en el resultado final. Está estrechamente relacionado con el concepto del caos.

Definición 1

Sensibilidad respecto condiciones iniciales Decimos que un sistema dinámico f definido en un dominio $\mathcal{M}$ es sensible respecto a condiciones iniciales si para cualquier $\delta > 0$ y punto $x \in \mathcal{M}$, podemos encontrar un punto y de forma que $d(x, y) < \delta$ y $d(\varphi_f(t; 0, x), \varphi_f(t; 0, y)) > e^{at}d(x, y)$ para algún $a > 0$.

A menudo, a un sistema con sensibilidad respecto a condiciones iniciales se lo llama un sistema con el **efecto mariposa**. Se conoce a este efecto gracias al matemático Edward Lorenz quien para explicar la sensibilidad respecto a condiciones iniciales usaba la siguiente metáfora: puede el aleteo de una mariposa en Brasil provocar un tornado en Texas (Lorenz, 1972). Aquí, el aleteo de la mariposa representa una muy pequeña variación de las condiciones iniciales y estas, provocan al final una gran diferencia representada por el tornado.

Dos conceptos relacionados con la sensibilidad respecto a condiciones iniciales son los conceptos de **transitividad topológica** y **densidad de órbitas periódicas**.

Definición 2

Transitividad topológica Decimos que un sistema f definido en un dominio $\mathcal{M}$ muestra transitividad topológica si para cualquier par de abiertos $U, V \in \mathcal{M}$ existe un tiempo t tal que $\varphi(t; 0, U) \cap V \neq \emptyset$.

En particular esto nos dice que dado un entorno de un punto del espacio, la evolución de dicho entorno con el tiempo acabará pasando por todos los puntos del espacio de fase $\mathcal{M}$.

Definición 3

Densidad de órbitas periódicas Decimos que un sistema f definido en un dominio $\mathcal{M}$ tiene órbitas periódicas densas si para cualquier punto $x \in \mathcal{M}$, existe un valor $\varepsilon > 0$ tal que existe un y con $d(x, y) < \varepsilon$ e y es una órbita periódica. Esto es, un punto y tal que $\varphi(t + \tau; t, y) = y$.

Un sistema denso en órbitas periódicas implica que no importa lo cerca que uno desee, siempre habrá una órbita periódica alrededor del punto. Sin embargo, estamos dejando libre el periodo de la órbita τ . Esto es, el tiempo que tarda la órbita en regresar al mismo punto.

Según R. Devaney (Devaney, 1989), estos tres conceptos que acabamos de ver, están estrechamente relacionados y forman la definición del Caos.

Definición 4

Caos Decimos que un sistema f definido en un dominio $\mathcal{M}$ es caótico si se verifica que:

- ▶ presenta transitividad topológica.
- ▶ es denso en órbitas periódicas.
- ▶ tiene sensibilidad respecto a condiciones iniciales.

La definición de Devaney del caos puede ser un tanto compleja de demostrar. Por este motivo, se definen los **exponentes de Lyapunov** que nos permita identificar rápidamente si un sistema es caótico o no.

Definición 5

Exponente de Lyapunov Definimos el exponente de Lyapunov λ como

$$\lambda = \limsup_{t \rightarrow \infty} \frac{1}{t} \log D\varphi_{x_0}(t; t_0, x_0)$$

Podemos interpretar el exponente de Lyapunov como la medida en la que se mueven los valores cercanos al punto x_0 . Si los exponentes de Lyapunov son todos negativos, entonces todas las direcciones alrededor de x_0 serán direcciones de acercamiento, y, por lo tanto, el sistema no mostrará sensibilidad respecto a condiciones iniciales. Sin embargo, si alguno es positivo, entonces tendrá una dirección de alejamiento y aparecerá la sensibilidad respecto a condiciones iniciales.

Con esto sobre la mesa, tenemos la necesidad de calcular la matriz $D\varphi_{x_0}$. Para ello vamos a recurrir a las ecuaciones variacionales. Estas ecuaciones nos permitirán no solo calcular los exponentes de Lyapunov, sino también expresar las variaciones a lo largo del tiempo de un punto cercano a un punto base.

Consideramos una ecuación diferencial:

$$\dot{\vec{x}} = f(t, \vec{x}).$$

Sabemos que, como $\varphi(t; t_0, \vec{x}_0)$ es solución a la ecuación diferencial, entonces verifica que:

$$\frac{d\varphi(t; t_0, \vec{x}_0)}{dt} = f(t, \varphi(t; t_0, \vec{x}_0)).$$

Ahora, derivando respecto de $\vec{x}_0$ tendremos

$$\frac{d}{d\vec{x}_0} \frac{d\varphi(t; t_0, \vec{x}_0)}{dt} = \frac{df(t, \varphi(t; t_0, \vec{x}_0))}{d\vec{x}} \frac{d\varphi(t; t_0, \vec{x}_0)}{d\vec{x}_0}.$$

Asumiendo que la regularidad del sistema es suficiente podemos intercambiar las derivadas en el lado izquierdo y aplicar la regla de la cadena en el lado derecho, de forma que nos queda la ecuación:

$$\frac{d}{dt} \frac{d\varphi(t; t_0, \vec{x}_0)}{d\vec{x}_0} = \frac{df(t, \varphi(t; t_0, \vec{x}_0))}{d\vec{x}} \frac{d\varphi(t; t_0, \vec{x}_0)}{d\vec{x}_0}.$$

Por lo que definiendo la matriz $Y = \frac{d\varphi(t; t_0, \vec{x}_0)}{d\vec{x}_0}$ nos queda la ecuación diferencial:

$$\dot{Y} = DfY.$$

Estas ecuaciones se conocen como **ecuaciones variacionales**. Serán de vital importancia siempre que queramos saber como se comportan puntos cercanos al punto que queremos describir. Fijémonos que $Y(t_0) = \frac{d\varphi(t_0; t_0, \vec{x}_0)}{d\vec{x}_0} = \frac{dx_0}{dx_0} = Id$. Por lo tanto, podemos escribir una ecuación diferencial extendida de la forma:

$$\begin{cases} \dot{x} = f(t, x), \\ \dot{Y} = Df_x(t, x)Y, \\ x(0) = x_0, \\ Y(0) = Id. \end{cases}$$

Esta ecuación será la que nos marque la evolución de valores cercanos a x_0 . Así pues, si tomamos un punto $x_0 + \xi$ tendremos una aproximación de la solución de la forma:

$$\varphi(t; t_0, x_0 + \xi) \approx \varphi(t; t_0, x_0) + Y\xi.$$

De esta forma, obtenemos una aproximación de Taylor de la solución que podemos utilizar para resolver ciertos problemas.

4.4 Simulink: herramientas para la representación de datos

Hemos visto que en un mismo sistema podemos encontrar diferentes soluciones en función del método de integración que empleemos. Será importante tener diferentes herramientas para visualizar y recoger los datos de la simulación. A lo largo de la asignatura hemos visto ya algunas herramientas como scope, XY Graph. En esta sección hablaremos de los detalles de configuración que puede tener las principales herramientas de la pestaña *Sinks*.

Herramientas de visualización gráfica

Las herramientas de visualización gráfica incluyen los elementos Scope, XY Graph y Floating Scope.

El primero de los elementos, Scope, nos permite representar la señal con respecto al tiempo. Podremos conectar varias señales y después realizar las configuraciones necesarias dentro del scope. Algunas propiedades importantes a tener en cuenta son:

- ▶ Configuración de propiedades: Accederemos a ello a través de la pestaña “*View→Configuration properties...*”. Aquí podremos establecer el número de entradas del bloque, así como las propiedades relacionadas con el eje x (el tiempo) y el eje y (*Display*). En la pestaña de *Display* podremos fijar el título de la gráfica así como los límites del eje y .
- ▶ Estilo de la gráfica: Accederemos a ello a través de la pestaña “*View→Style...*”. Aquí podremos configurar el tipo de gráfica, colores, y el resto de propiedades relativas al fondo.
- ▶ Activar la leyenda: Podemos activar la leyenda de la gráfica. Esto es especialmente interesante para volcar los datos a una memoria.
- ▶ Opciones de exportación: Podemos exportar el gráfico generado al portapapeles para pegarlo en un documento o usar la opción de *Print* para generar un archivo.

El elemento XY Graph nos permite realizar trabajar como con un bloque Scope pero en el que el valor de x está definido por una señal. En este bloque tendremos que fijar los límites del gráfico. Cuando ejecutemos la simulación se nos abrirá una nueva ventana con la gráfica generada.

Finalmente, el elemento Floating Scope nos permite añadir un elemento de tipo Scope de forma que no esté conectado. Esto puede ser útil para mantener el diseño del modelo limpio de señales y seguir obteniendo información sobre el resultado. Debemos enlazar la señal al Floating Scope a través del propio elemento. Cuando hagamos una doble pulsación se nos abrirá un *scope* vacío. Podemos enlazar una o más señales pulsando en el desplegable que acompaña al candado y seleccionar “*Signal selector ...*”. Esto nos habilitará el modelo y podremos seleccionar aquellas señales que nos interesan para incluirlas en la gráfica.

En el vídeo “Depuración y Data logging en Simulink” profundizaremos sobre las propiedades que podemos configurar en los bloques de Scope de Simulink para obtener la mejor información posible.



Accede al vídeo: Depuración y Data logging en Simulink

Herramientas de salidas de datos

En este segundo bloque de herramientas tenemos las herramientas que nos permiten visualizar los valores exactos de la simulación. Los principales bloques son el Display, para visualizar un único valor; To File y To Workspace, para escribir los valores en un documento o pasar datos al *Workspace* de Matlab.

El bloque Display es de uso bastante directo. Le conectaremos una señal y nos dará el valor de esa señal. Podremos configurar el tipo de dato que mostraremos.

El bloque To File nos permite definir un fichero de datos de Matlab. En la configuración deberemos establecer el nombre del fichero en el que pondremos los datos y el nombre de la variable que contendrá los datos. El bloque To Workspace permite traspasar los datos de la señal al Workspace de Matlab para poder utilizarlos a nuestra conveniencia. En ambos casos se genera una estructura llamada out que contiene todas las variables definidas en los bloques correspondientes. Esto nos resultará útil para realizar representaciones gráficas más complejas o realizar ciertos análisis de las señales obtenidas.

Herramientas de terminación

El último tipo de bloques que nos pueden interesar son aquellos bloques que terminan una señal sin hacer nada con ella. El ejemplo básico es el bloque Terminator que permite poner una salida a la nada. Esto nos interesará cuando la salida de un bloque no nos interese. Como ya hemos comentado anteriormente, una señal no puede quedar conectada. Si no nos interesa la tendremos que conectar a este tipo de bloques.

El segundo tipo de bloque que podremos usar es el bloque Stop Simulation. Este bloque permite parar una simulación antes de completar el tiempo predefinido. Por

ejemplo, debido a que se han cumplido ciertas condiciones finales. La simulación continuará siempre que la señal de entrada sea 0.

4.5 Modelos en Simulink

Veamos en esta última sección algunos ejercicios que podemos considerar con lo que hemos visto en este tema. Vamos a tratar:

- El sistema de Lorentz, que presenta valores para los que se observa el caos.
- El problema de la bala de cañón con condiciones de parada.

Un sistema caótico

Ejercicio 2. Realiza un modelo en Simulink para describir el modelo conocido como las ecuaciones de Lorentz descrito por la ecuación:

$$\begin{cases} \dot{x} = \sigma(y - x), \\ \dot{y} = x(\rho - z) - y, \\ \dot{z} = xy - \beta z. \end{cases}$$

Con valores $\sigma = 10$, $\rho = 14$ y $\beta = 8/3$ y condiciones iniciales $x_0 = y_0 = z_0 = 1$.

Las ecuaciones de Lorentz describen un modelo para la convección atmosférica. La variable x es proporcional a la tasa de convección, y a la variación de temperatura horizontal y z a la variación de temperatura vertical. Las constantes σ , ρ y β son parámetros proporcionales al número de Prandtl, el número de Rayleigh y una cierta dimensión física relacionada con la capa de convección.

El sistema ya está escrito en la forma que nos interesa. Para pasarlo a Simulink vamos a introducir tres bloques de integración y asignaremos los nombres tal y como se muestra en la figura 6.

Además, introducimos los bloques necesarios para modelar la ecuación:

- Cuatro bloques de resta.

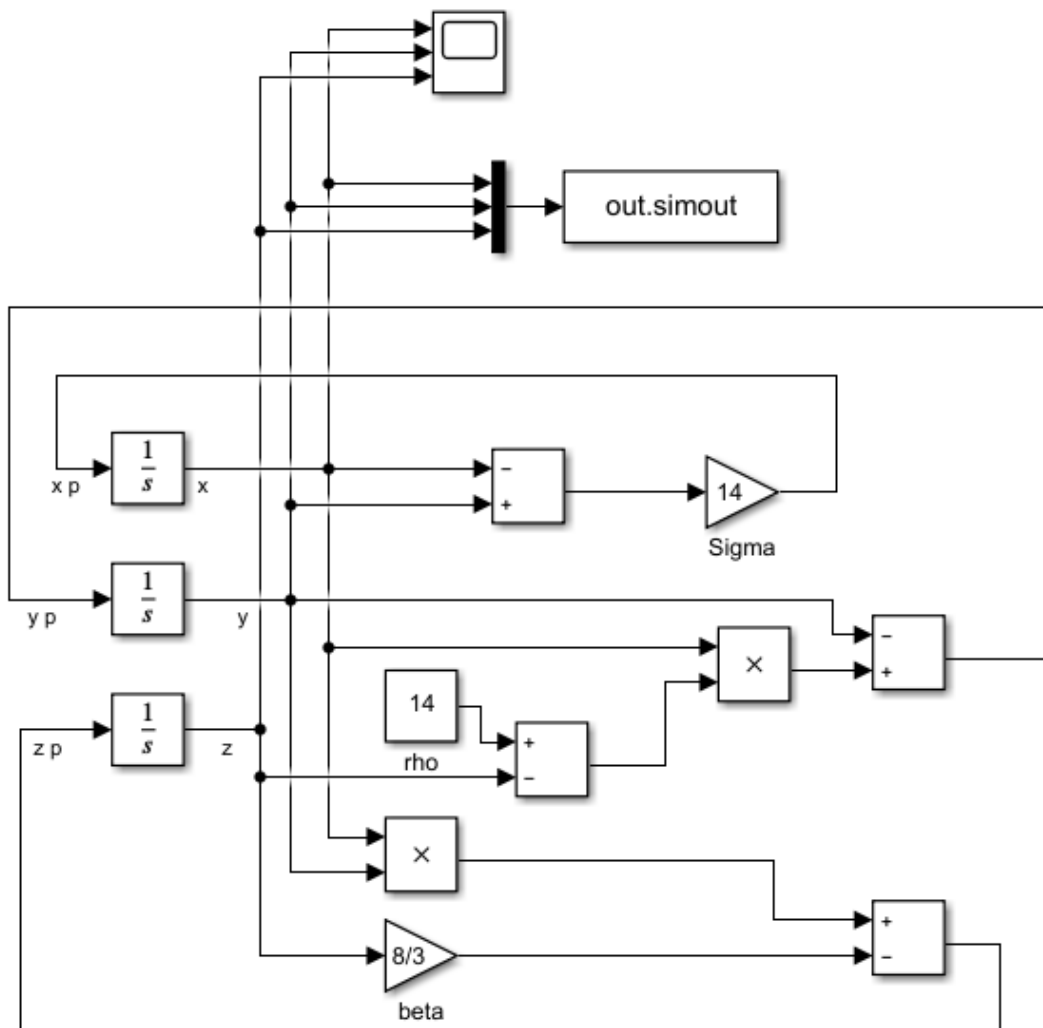


Figura 6: Modelo de Lorentz completo.

- Dos bloques de producto por escalar para los productos de σ y β .
- Dos bloques de producto.
- Un bloque de constante para ρ .

Una vez introducidos los elementos, realizamos las conexiones necesarias tal y como se muestra en la Figura 6 teniendo en cuenta que el resultado del lado derecho se conectará al lado izquierdo.

Para visualizar la salida de los datos en la Figura 6 se ha empleado un bloque scope para las tres variables tal y como se muestra en la Figura 7 derecha. Como la naturaleza del sistema es 3-dimensional, también nos interesa visualizar las 3 variables simultáneamente en $\mathbb{R}^3$. Simulink no permite la ejecución de este tipo de gráficos. Sin

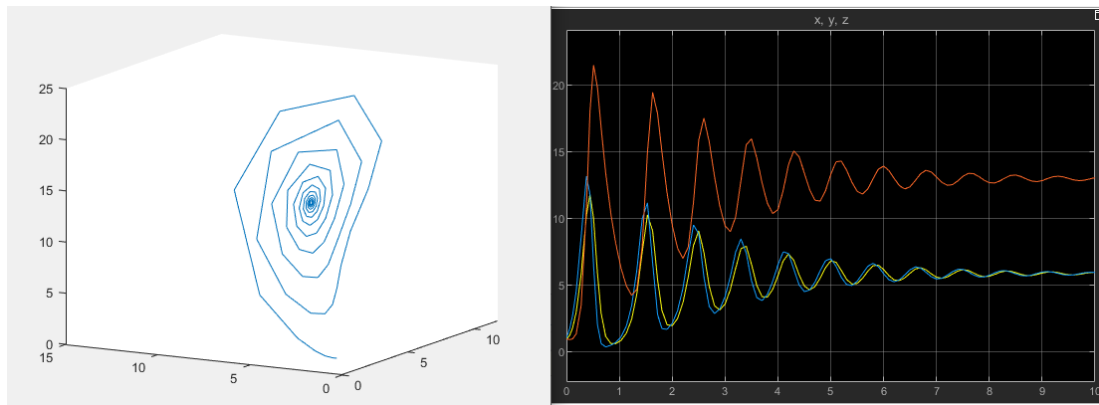


Figura 7: Resultados de la simulación en el modelo de Lorentz con los parámetros dados en el enunciado.

embargo, podemos exportar los datos al Workspace de Matlab y realizar las representaciones gráficas correspondientes. Para ello, hemos empleado la combinación de los bloques:

- Mux que nos permite juntar tres señales en forma de vector y se ha empleado un bloque y encontraremos en la pestaña *Signal Routing*.
- To Workspace de la pestaña *Sinks* que nos permite exportar datos al Workspace de Matlab. Una vez ejecutado el modelo podemos acudir a la ventana de comandos de Matlab y tendremos creada la variable `out`. Esta variable es una estructura que dentro contendrá por defecto los intervalos de tiempo de la simulación. Al añadir el bloque To Workspace también contendrá el valor de la señal en cada instante de tiempo. En nuestro caso hemos configurado el bloque para que el nombre de la variable de salida sea `xyz`. Así pues, en la ventana de comandos ejecutaremos:

```
xyz=out.xyz_vals; %Recupera xyz
plot3(xyz(:,1),xyz(:,2),xyz(:,3)) %dibujo 3D
```

para obtener el gráfico tal y como se muestra en la Figura 7 izquierda. Podemos observar que la trayectoria tiende a un punto fijo realizando una espiral.

Las ecuaciones de Lorentz describen un sistema que, para ciertos valores de los parámetros es caótico. Esto lo podemos ver si tomamos los parámetros $\rho = 28$, $\sigma = 10$ y $\beta = 8/3$. En la Figura 8 podemos ver la representación gráfica de dos soluciones cuyas condiciones iniciales son $(1, 1, 1)$ en naranja y $(1.0001, 1, 1)$ en azul. Podemos ver como inicialmente la solución coincide (imagen superior izquierda). Si seguimos con la simulación (imagen superior derecha) las dos órbitas se van separando poco a poco

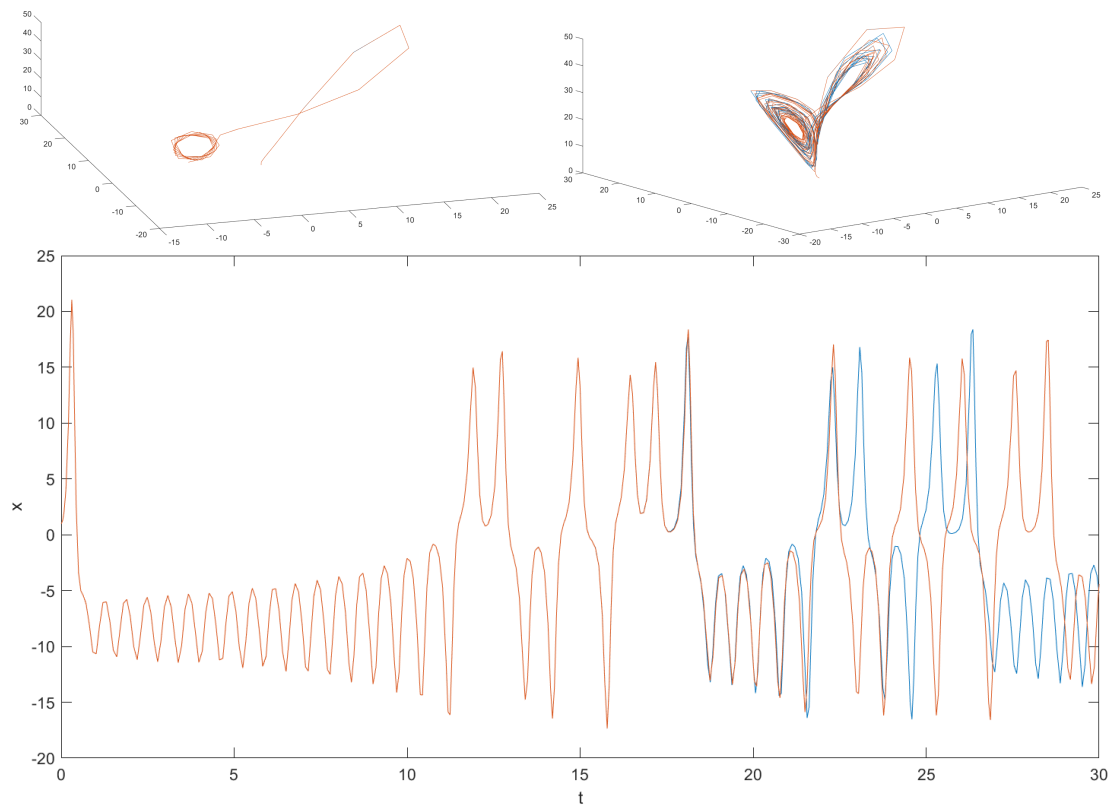


Figura 8: Resultados de la simulación en el modelo de Lorentz con parámetros $\rho = 28$, $\sigma = 10$ y $\beta = 8/3$ para la condición inicial $(1, 1, 1)$ en naranja y $(1.0001, 1, 1)$ en azul. La imagen superior izquierda muestra la simulación durante las primeras 3 unidades de tiempo, la superior derecha durante las primeras 30 unidades de tiempo. La imagen inferior muestra la evolución de la coordenada x a lo largo de las 30 unidades de tiempo simuladas.

hasta llegar a un punto en el que ya no hay ninguna similitud. En la imagen inferior podemos ver la evolución de la coordenada x para cada una de las dos simulaciones. Se puede apreciar como, a partir del tiempo $t = 18$ las dos trayectorias se van separando hasta llegar a una evolución completamente diferente.

Detección de condiciones de parada: la bala de cañón en el suelo

Ejercicio 3. Considera el problema del movimiento de una bala de cañón moviéndose sujeta a la gravedad y a la fricción del aire que hemos visto en temas anteriores.

Determina el momento en el que la bala colisionará con el suelo ($y = 0$).

Para realizar este ejercicio tomaremos el modelo que hemos construido en el tema

anterior. A continuación añadiremos los siguientes elementos:

- ▶ Un bloque `stop` para parar la simulación.
- ▶ Un bloque `Relational Operator` que encontraremos en *Commonly Used Blocks* y en *Logic and Bit Operations*. Este bloque nos dará una señal lógica (0 o 1) dependiendo de la comparación que configuremos. En nuestro caso queremos mandar una señal de 1 al bloque `stop` cuando la altura y sea negativa. Luego pondremos el comparador a $<$.
- ▶ Un bloque constante configurado a 0.
- ▶ Tres bloques de `Display` para obtener el resultado.
- ▶ Un bloque de entrada de tiempo `Clock` que se encuentra en *Sources*, para determinar el tiempo de contacto con el suelo.

Deberemos realizar las conexiones como se muestra en la Figura 9. En esta figura ya se ha ejecutado la simulación. Podemos observar como los tres bloques `Display` nos muestran la solución al problema propuesto. Tenemos que la bala tardará 3.922 segundos y recorrerá 50.71 metros antes de llegar al suelo. La altura en y aparece como $-7.017 \cdot 10^{-14}$. Observemos que se trata de una cifra pequeña y negativa. Simulink detecta el valor más cercano posible a 0 y entonces pausa la simulación. Si debido a los pasos de tiempo del método de integración el valor se pasa del 0, Simulink retrocede hasta dar con la mejor aproximación posible.

Observación: En este modelo hemos decidido incluir la comparación de forma general, con un bloque `Constant` y un bloque `Relational Operator`. Esto nos permitirá comparar dos señales cualesquiera. Por ejemplo, determinar si en algún momento la variable x es mayor que la variable y . Si únicamente queremos comparar con un escalar, entonces podemos emplear el bloque `Compare To Zero` o `Compare To Constant` que realizan el mismo efecto en un único bloque.

4.6 Referencias bibliográficas

Devaney, R. L. (1989). *An Introduction to Chaotic Dynamical Systems*. Addison-Wesley.

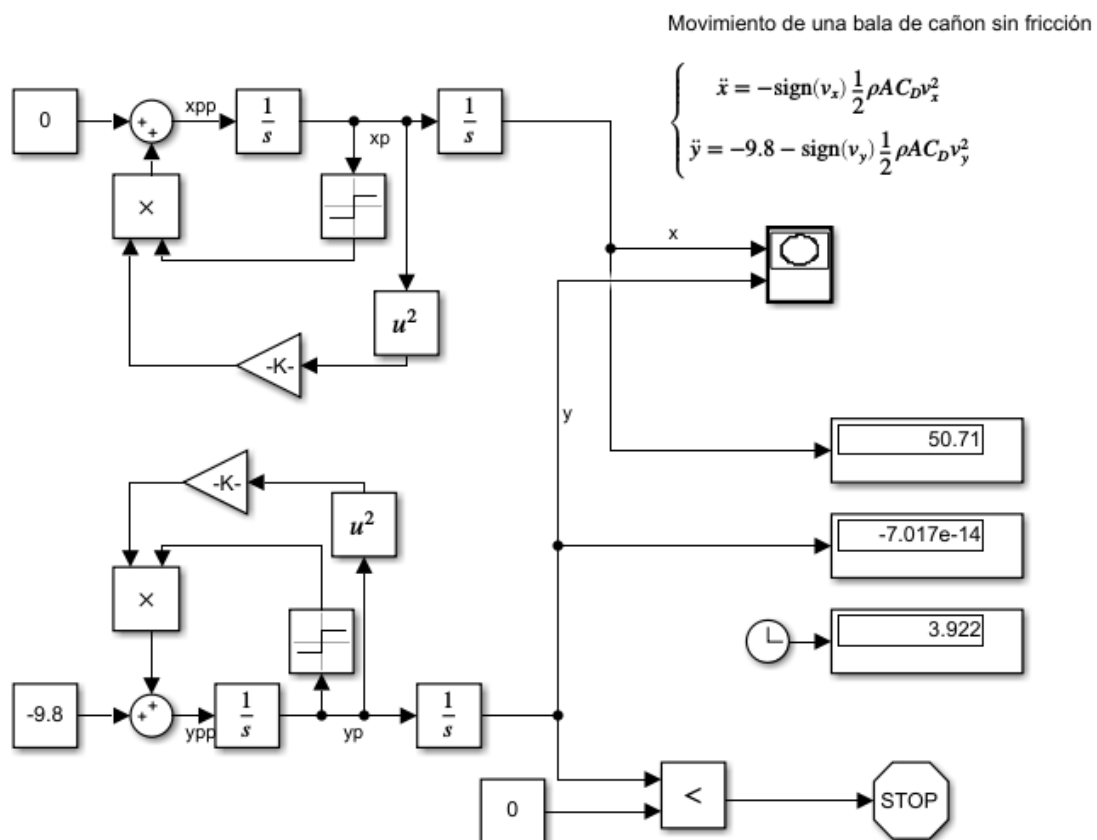


Figura 9: Modelo y resultados del modelo para una bala de cañón con detección de colisión al llegar al suelo en $y = 0$.

Lorenz, E. N. (1972). Predictability; does the flap of a butterfly's wings in brazil set off a tornado in texas? *AAAS Section on Environmental Sciences New Approaches to Global Weather*.